1)

```
// Add test case for a toggle coverage closure
A=4'b1000; B=4'b1110;
check_result(5'b10110);

//Toggle closure for reset
assert_reset;
```

2) **Bugs:** valid output is not considered in the reset. Missing begin-end for else branch. Coverage exclusions are not justified.

Label	Description	Stimulus Generation	Functional Coverage (Later)	Functionality Check
ENCODER_1	When the reset is asserted, the outputs Y and valid must be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
ENCODER_2	Tyring all possible valid values of D input (4'b0001 to 4'b1111)	Directed during the simulation	-	A checker in the testbench with each D input change to make sure the output is correct

```
module priority_enc_tb;
logic  clk;
logic  rst;
logic  [3:0] D;
logic  [1:0] Y;
logic  valid;
integer error_count;
integer correct_count;

priority_enc DUT (.*);

// Create the clock and reset
initial begin
   clk = 0;
   forever #1 clk = !clk;
end

// Create the Opcode and A and B
initial begin
   error_count = 0;
   correct_count = 0;
   D = 0;

   assert_reset;

   D = 4'b1111;
   check_result(2'b11, 1'b1);
   D = 4'b1110;
   check_result(2'b10, 1'b1);
   D = 4'b1101;
   check_result(2'b11, 1'b1);
   D = 4'b1100;
   check_result(2'b01, 1'b1);
   D = 4'b1011;
   check_result(2'b11, 1'b1);
   D = 4'b1010;
   check_result(2'b10, 1'b1);
   D = 4'b1001;
   check_result(2'b11, 1'b1);
   D = 4'b1000;
   check_result(2'b00, 1'b1);
   D = 4'b0111;
   check_result(2'b11, 1'b1);
   D = 4'b0110;
   check_result(2'b10, 1'b1);
   D = 4'b0101;
   check_result(2'b11, 1'b1);
   D = 4'b0100;
   check_result(2'b01, 1'b1);
   D = 4'b0011;
   check_result(2'b11, 1'b1);
   D = 4'b0010;
   check_result(2'b10, 1'b1);
   D = 4'b0001;
   check_result(2'b11, 1'b1);
   D = 4'b0000;
   check_result(Y, 1'b0);

   // Coverage closure
   assert_reset;

   $display("%t: At end of test err
   $stop;
end
```

Karim Waseem

```
task check_result(logic [1:0] expected_Y, logic expected_valid);
   @(negedge clk);

   if (expected_Y != Y) begin
      error_count = error_count + 1;
      $display("%t: Error: Y should equal 0d%0d but the calcuated result is 0d%0d ", $time, expected_Y, Y);
   end
   else
      correct_count = correct_count + 1;

   if (expected_valid != valid) begin
      error_count = error_count + 1;
      $display("%t: Error: valid should equal 0d%0d but the calcuated result is 0d%0d ", $time, expected_valid, valid);
   end
   else
      correct_count = correct_count + 1;
endtask // check_result

task assert_reset;
   rst = 1;
   // Check reset value
   check_result(2'b0, 1'b0);
   rst = 0;
endtask // assert_reset
endmodule
```

3) Coverage exclusion for default branch is acceptable.

Label	Description	Stimulus Generation	Functional Coverage (Later)	Functionality Check
ALU_1	When the reset is asserted, the output C value must be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
ALU_2	Verifying maximum negative value on A and maximum negative value on B while opcode is Add	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_3	Verifying maximum negative value on A and maximum negative value on B while opcode is Sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_4	Verifying maximum negative value on A and zero value on B while opcode is Add	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_5	Verifying maximum negative value on A and zero value on B while opcode is Sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_6	Verifying maximum negative value on A and maximum positive value on B while opcode is Add	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_7	Verifying maximum negative value on A and maximum positive value on B while opcode is Sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_8	Verifying maximum negative value on A and maximum positive value on B while opcode is Add	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_9	Verifying maximum negative value on A and maximum positive value on B while opcode is Sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_10	Verifying zero on A and maximum negative value on B while opcode is Add	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_11	Verifying zero on A and maximum negative value on B while opcode is Sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_12	Verifying zero on A and zero on B while opcode is Add	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_13	Verifying zero on A and zero on B while opcode is Sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_14	Verifying zero on A and maximum positive value on B while opcode is Add	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_15	Verifying zero on A and maximum positive value on B while opcode is Sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_16	Verifying maximum positive value on A and maximum negative value on B while opcode is Add	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_17	Verifying maximum positive value on A and maximum negative value on B while opcode is Sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_18	Verifying maximum positive value on A and zero on B while opcode is Add	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_19	Verifying maximum positive value on A and zero on B while opcode is Sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_20	Verifying maximum positive value on A and maximum positive value on B while opcode is Add	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_21	Verifying maximum positive value on A and maximum positive value on B while opcode is Sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_22	Set B to non-all 0 and non-all 1, and apply all 0 to port A while opcode is Not_A	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_23	Set B to non-all 0 and non-all 1, and apply all 1 to port A while opcode is Not_A	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_24	Apply all-0, all-1, and walking ones to port B while setting port A to a value that yield the opposite results. Opcode: ReductionOR_B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_25	Set A and B to a constant values and loop over all opcodes	Directed during the simulation	-	A checker in the testbench to make sure the output is correct

Kareem Waseem

```
module ALU_tb;
// inputs to instantiated ALU
reg clk, reset;
reg  signed [3:0] A; // Input data A: 2's complement
reg  signed [3:0] B; // Input data B: 2's complement

// Output of ALU
wire signed [4:0] C; // ALU output: 2's complement

integer       error_count; // 32-bit signed

integer       correct_count; // 32-bit signed

localparam Add = 2'b00,
      Sub = 2'b01,
      Not_A = 2'b10,
      ReductionOR_B = 2'b11;

localparam       MAXPOS = 7,
                 ZERO   = 0,
                 MAXNEG = -8;

reg [1:0] Opcode; // The opcode

ALU_4_bit DUT (.*);

// Create the clock and reset

initial begin
   clk = 0;
   forever #1 clk = !clk;
end

// Create the Opcode and A and B
initial begin
   error_count = 0;
   correct_count = 0;
   A = 0;
   B = 0;
   Opcode = 0;

   assert_reset;

   // Apply all permutations of max pos, max neg, and 0 to each opcode
   A=MAXNEG; B=MAXNEG;
   Opcode = Add;
   check_result(Opcode, A, B, -16);
   Opcode = Sub;
   check_result(Opcode, A, B, ZERO);

   A=MAXNEG; B=ZERO;
   Opcode = Add;
   check_result(Opcode, A, B, MAXNEG);
   Opcode = Sub;
   check_result(Opcode, A, B, MAXNEG);

   A=MAXNEG; B=MAXPOS;
   Opcode = Add;
   check_result(Opcode, A, B, -1);
   Opcode = Sub;
   check_result(Opcode, A, B, -15);

   A=ZERO; B=MAXNEG;
   Opcode = Add;
   check_result(Opcode, A, B, MAXNEG);
   Opcode = Sub;
   check_result(Opcode, A, B, 8);
```

```
   A=ZERO; B=ZERO;
   Opcode = Add;
   check_result(Opcode, A, B, ZERO);
   Opcode = Sub;
   check_result(Opcode, A, B, ZERO);

   A=ZERO; B=MAXPOS;
   Opcode = Add;
   check_result(Opcode, A, B, MAXPOS);
   Opcode = Sub;
   check_result(Opcode, A, B, -7);

   A=MAXPOS; B=MAXNEG;
   Opcode = Add;
   check_result(Opcode, A, B, -1);
   Opcode = Sub;
   check_result(Opcode, A, B, 15);

   A=MAXPOS; B=ZERO;
   Opcode = Add;
   check_result(Opcode, A, B, MAXPOS);
   Opcode = Sub;
   check_result(Opcode, A, B, MAXPOS);

   A=MAXPOS; B=MAXPOS;
   Opcode = Add;
   check_result(Opcode, A, B, 14);
   Opcode = Sub;
   check_result(Opcode, A, B, ZERO);

   // Set B to non-all 0 and non-all 1.  Apply all 0 and all 1's to bitwise invert input A opcode.
   Opcode = Not_A;
   A=4'b0000; B=4'b0101;
   check_result(Opcode, A, B, 5'b11111);
   A=4'b1111; B=4'b1010;
   check_result(Opcode, A, B, 5'b00000);

   // Apply 0, all 1's, and walking 1's to B for ReductionOR_B opcode. Set A to a value that will yield the opposite result.
   Opcode = ReductionOR_B;
   A=4'b1111; B=4'b0000;
   check_result(Opcode, A, B, 0);
   A=4'b0000; B=4'b1111;
   check_result(Opcode, A, B, 1);
   A=4'b0000; B=4'b0001;
   check_result(Opcode, A, B, 1);
   A=4'b0000; B=4'b0010;
   check_result(Opcode, A, B, 1);
   A=4'b0000; B=4'b0100;
   check_result(Opcode, A, B, 1);
   A=4'b0000; B=4'b1000;
   check_result(Opcode, A, B, 1);

   // Set the operands and walk through the opcodes
   A=4'b0101; B=4'b1010;
   Opcode = Add;
   check_result(Opcode, A, B, -1);
   Opcode = Sub;
   check_result(Opcode, A, B, 11);
   Opcode = Not_A;
   check_result(Opcode, A, B, 5'b11010);
   Opcode = ReductionOR_B;
   check_result(Opcode, A, B, 1);

   // Coverage closure for reset
   assert_reset;

   $display("%t: At end of test error count is %0d and correct count = %0d", $time, error_count, correct_count);
   $stop;
end
```

```
task check_result (input [1:0] Opcode, input signed [3:0] A, B, input signed [4:0] expected_result);
   @(negedge clk);
   if (expected_result !== C) begin
     error_count = error_count + 1;
     $display("%t: Error: For Opcode = %0b, A= 0d%0d, and B=0d%0d C should equal 0d%0d but is 0d%0d ", $ti
   end
   else
     correct_count = correct_count + 1;
endtask

task checkC(input signed [4:0] expected_C);
   @(negedge clk);
   if (expected_C !== C) begin
     error_count = error_count + 1;
     $display("%t: Error: C should equal 0d%0d but the calcuated result is 0d%0d ", $time, expected_C, C);
   end
   else
     correct_count = correct_count + 1;
endtask // checkC

task assert_reset;
   reset = 1;
   // Check reset value of C
   checkC(5'b0);
   reset = 0;
endtask // assert_reset
endmodule
```

4) **Bug**: post-adder has extra delay stage. Also, C_reg was not included in the reset. Toggle Coverage exclusion for mult_out [47:36] is acceptable.

Label	Description	Stimulus Generation	Functional Coverage (Later)	Functionality Check
DSP_1	When the reset is asserted, the output P must be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
DSP_2	Verify the normal functionality of the DSP out of reset	Randomized during the simulation	-	A checker in the testbench comparing the P output with the golden model output

```
module DSP_tb;

   logic  [17:0] A, B, D, adder_out;
   logic  [47:0] C;
   logic clk, rst_n;
   logic  [47:0] P, mult_out, P_exp;

   integer       error_count; // 32-bit
   integer       correct_count; // 32-bit

   DSP DUT(.*);

   // Create the clock
   initial begin
      clk = 0;
      forever #1 clk = !clk;
   end

   // Stimulus Generation
  initial begin
      error_count = 0;
      correct_count = 0;
      A = 0;
      B = 0;
      C = 0;
      D = 0;

      assert_reset;

      //Randomization
      repeat(10000) begin
         A = $random();
         B = $random();
         C = $random();
         D = $random();
         // Golden Model
         adder_out = B + D;
         mult_out =  adder_out * A;
         P_exp = mult_out + C;
         // Check output
         check_result;
      end
      check_reset;

      $display("%t: At end of test error count is %
      $stop;
   end
```

Karim Wasem

```
   // Checker
   task check_result();
      repeat(4) @(negedge clk);
      if (P_exp !== P) begin
        error_count = error_count + 1;
        $display("%t: Error: For A= 0d%0d, B=0d%0d,
      end
      else
        correct_count = correct_count + 1;
   endtask

   task check_reset;
      //assert reset
      rst_n = 0;
      check_0;
      //desert reset
      rst_n = 1;
   endtask

   task check_0;
      @(negedge clk);
      if (P !== 0) begin
        error_count = error_count + 1;
        $display("%t: Error: Reset value is asserte
      end
      else
        correct_count = correct_count + 1;
   endtask

endmodule
```

Kareem Waseem